PROJECT BRIEFING GUIDE

# Counterexample-Guided Auditing and Constrained LLM Repair of Learned Rewards for Protein Fitness Optimization

*Everything you need to know, from absolute zero, written for the two of us.*

| Team | Varun Aditya (1RV24IS143) and Ravi R. Naidu (1RV25IS409) |
|---|---|
| **Course** | IS353IA Artificial Intelligence and Machine Learning, Lab Part B |
| **Class** | 5th Semester, ISE B, Lab Batch B3 |
| **Department** | Information Science & Engineering |
| **Prepared** | 2 October 2026 |

## How to read the coloured boxes

Every claim in this guide carries one of these labels, so you always know how much to trust it.

| | |
|---|---|
| KEY IDEA | The one thing to remember from a section. |
| VERIFIED FACT | Checked against a paper or web page. Source numbers are in brackets, for example [3]. Full list in Part 7. |
| OUR DESIGN CHOICE | A decision WE made in the project plan. It is not a fact from literature and we can change it. |
| OPEN DECISION | Not decided yet. We have to decide together. |
| WATCH OUT | A mistake that is easy to make, or a limit we must state honestly. |
| TRY IT | A small exercise or snippet to make the idea concrete. |

# Roadmap: how this guide is organised

The project sits where four topics meet: protein data, machine-learning predictors, search algorithms and language models. You do not need a biology background. Read Part 1 first, then Part 2 in order, then dip into the rest as you work.

| Part | What it covers | After reading you can… |
|---|---|---|
| **1 The project in two minutes** | The problem, the idea, the four-stage loop, scope and novelty | Explain the project to a friend in one minute |
| **2 Foundations from zero** | Proteins, fitness data, prediction models, uncertainty, search, reward hacking, LLM checking, distribution shift | Understand every word used in the synopsis |
| **3 The method in detail** | Reward formula, five failure detectors, repairs, verification, worked example | Describe exactly what the code has to do |
| **4 Data and experiments** | The datasets, the hidden-label simulation, baselines, metrics, statistics | Design a fair experiment and spot unfair ones |
| **5 Plan and first steps** | 18-week schedule, learning path, first-week exercise, risks, tools | Start working on Monday |
| **6 Honesty check** | What is sourced, what is our choice, what is open, and where the submitted synopsis needs updating | Never overclaim in front of the professor |
| **7 Viva prep, glossary, sources** | Likely questions with grounded answers, A to Z terms, reading list | Prepare for the evaluation and read the right papers |

**KEY IDEA**

This is an **ML methodology project**. Protein data is the testbed. The question we study is: *when a learned score misleads a search, can we detect it, work out why, and repair the score safely?*

We do **not** claim to design real proteins. That sentence is worth memorising.

PART 1

# The project in two minutes

What we are doing, why it matters, and what is (and is not) new

## 1.1 The short version

Researchers who engineer proteins want versions of a protein that work better, for example more stable or more active. Measuring one version needs a lab experiment, which is slow and expensive. So they train a machine-learning model to **predict** how good a version is, and then let a search algorithm hunt for versions the model scores highly.

Here is the trap. A model is only trustworthy near the examples it was trained on, and a search algorithm deliberately walks away from those examples. It ends up in places where the model is badly wrong but happens to give high scores. The score the search sees keeps rising while the real quality does not.

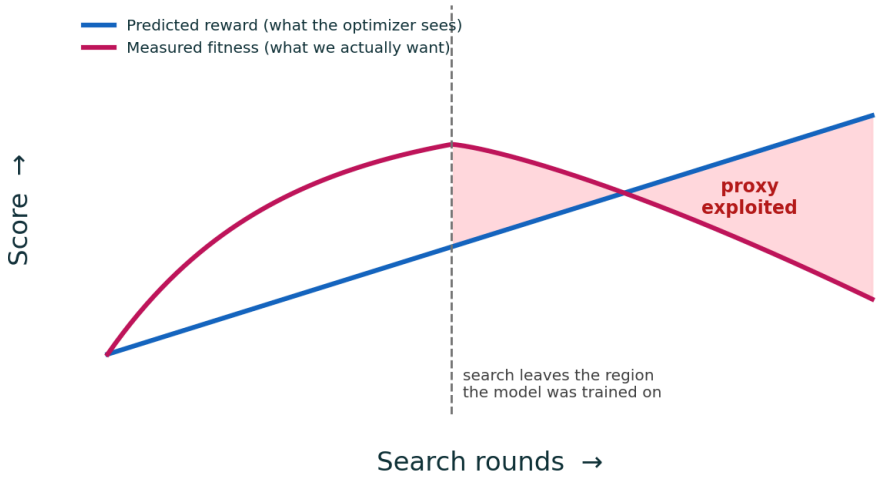



*Figure 1.1 The problem in one picture. This is a schematic sketch to explain the idea. It is not experimental data.*

Our project builds a **safety net around the search**. It is a loop with four stages:

1. **Specify.** The scoring rule (the "reward") is written down in a strict, checkable format. A language model helps write it, a rule-based validator checks it.
2. **Optimize.** A search picks a batch of protein versions to "measure", using the current reward.
3. **Falsify.** The true measurements for that batch are revealed. Five detectors check whether the truth contradicts what the reward expected. A contradiction is called a **counterexample**.
4. **Repair.** The system works out the type of failure, tries the smallest fitting fix from a fixed list, verifies the fix, then continues, rolls back, or stops.

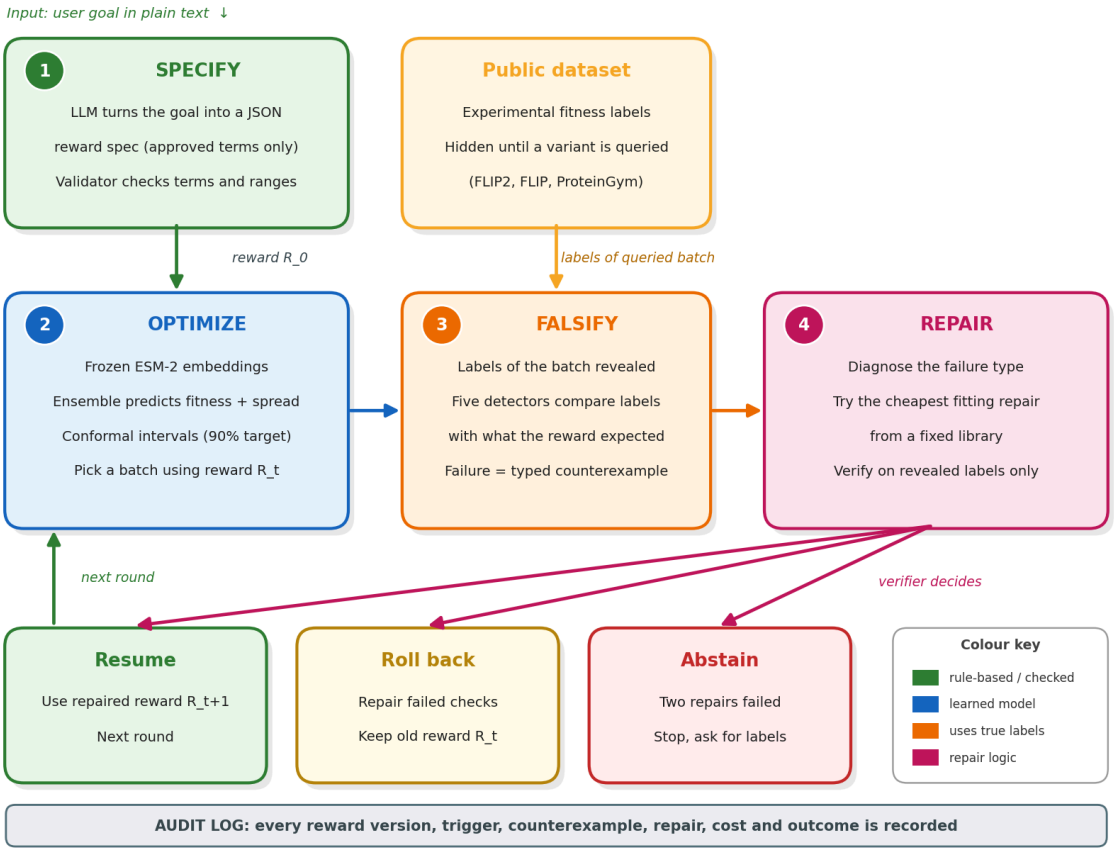



*Figure 1.2 The four-stage loop (our architecture diagram, Figure 5.1 in the synopsis).*

ANALOGY

Imagine studying for an exam using a friend's answer key. If you only try to match the key, you also learn its mistakes. A good student notices when the key disagrees with the textbook, works out what kind of mistake it is, corrects only that part, and stops trusting the key if it keeps being wrong.

Our loop is that good student.

## 1.2 The name of the idea, word by word

| Word in the title | Plain meaning |
|---|---|
| **Counterexample-guided** | Repairs are triggered by concrete cases where truth contradicts the reward, not by a timer. |
| **Auditing** | Every reward version, trigger, repair and outcome is logged so anyone can trace what happened. |
| **Constrained LLM** | A language model is used, but only to fill a strict JSON form and to choose repairs from a fixed menu. It cannot invent new reward terms or write code. |
| **Repair** | Change the reward by the smallest allowed amount, then verify the change. |
| **Learned rewards** | The score comes from a trained ML model, not from a formula someone derived by hand. |
| **Protein fitness optimization** | Searching for protein variants with higher measured fitness. |

## 1.3 Scope: what is in and what is out

| In scope | Out of scope |
|---|---|
| Retrospective experiments on public experimental datasets (FLIP2, original FLIP, optionally ProteinGym), with labels hidden until queried | Wet-lab work or claims that any variant is functional, safe or newly engineered |
| Frozen ESM-2 embeddings, an ensemble of simple regressors, conformal intervals | Training or fine-tuning a protein language model |
| Five failure detectors, a fixed repair library, a verifier, rollback, abstention, audit log | Free-form reward-code writing by the LLM |
| Baselines: random, greedy, genetic, AdaLead-style, Bayesian optimization, fixed penalty, retraining | Digital twins, CRISPR guide design and de novo protein generation (ideas we considered and dropped earlier) |
| Reinforcement learning only as an optional extra baseline | Making RL the main method (unless our professor requires it, see Part 6) |

## 1.4 What is new and what is borrowed

Honesty here protects us in the viva. Almost every ingredient already exists. The contribution is the way they are combined and tested.

| Borrowed from earlier work | Proposed in this project |
|---|---|
| Protein benchmarks and splits [1][2][3] | Treating a label that contradicts the reward as a **typed counterexample** to an explicit reward specification |
| ESM-2 embeddings [4], ensembles [16], conformal intervals [15] | A **repair library** with a repair cost, so the "smallest" fix is measurable |
| Uncertainty and distance penalties in the reward [9][11] | **Verification, rollback and abstention** rules for each repair |
| Greedy, genetic, AdaLead-style and Bayesian search [12][14] | A versioned **audit log** of every reward change |
| LLMs that write reward code for RL [7][8] | A **recovery-focused evaluation**: how fast does the search get back on track after a failure? |

**WATCH OUT**

Say "in the papers we reviewed, we did not find a protocol that does all of this". Never say "this has never been done" or "we are the first". A literature search cannot prove that something does not exist.

PART 2

# Foundations from zero

Eleven small building blocks. Each one is used somewhere in the project.

Take these in order. Each block has the same shape: what it is, a tiny example, and why it matters to us.

## 2.1 Proteins are strings, and a "variant" is a string with edits

A protein is a chain of amino acids. There are 20 standard types [W2] and each has a one-letter code, so a protein sequence can be stored as a string such as `MKTAYIAK`. The natural version found in nature is called the **wild type**. A **mutation** replaces one amino acid with another. A protein with one or more mutations is a **variant**.

To count how different two variants are we use the **Hamming distance**: the number of positions at which two equal-length strings differ [W4]. It only counts substitutions, not insertions or deletions [W4b]. Our datasets contain substitution variants of one protein, so Hamming distance is a natural measure of "how many mutations apart".

| Sequence | What it is | Mutations from wild type |
|---|---|---|
| `MKTAYIAK` | Wild type | 0 |
| `MKTAYIAR` | Variant 1 (K⟶R at position 8) | 1 |
| `MQTAYIGR` | Variant 2 (K⟶Q at 2, A⟶G at 7, K⟶R at 8) | 3 |

*Hamming distance between Variant 1 and Variant 2 = 2 (they differ at positions 2 and 7). This small example is made up by us for teaching.*

> **KEY IDEA**
> Everything in this project works on strings of letters plus one number per string (its measured fitness). You do not need to understand protein chemistry to do the project.

## 2.2 Fitness, fitness landscapes and where the data comes from

**Fitness** is a measured number describing how well a variant performs some function, for example binding strength, enzyme activity or thermostability. A **fitness landscape** maps each sequence to its functional output [W2]. Imagine a mountain range where the position is the sequence and the height is the fitness. Protein engineering is the search for high peaks.

Data comes from experiments such as **deep mutational scanning (DMS)**. DMS measures the functional consequences of thousands of mutations in a single experiment: build a large library of variants, apply a selection pressure, count each variant before and after with DNA sequencing, and turn the change in counts into a fitness score [W2]. Some FLIP2 datasets come from related screens, for example error-prone PCR libraries measured in a microfluidic screen (IRED) [W1].

**VERIFIED FACT** [W3]

The effect of combining mutations is not always the sum of the single effects. This is called **epistasis**, and it makes landscapes rugged. The space of possible sequences is also exponentially large, so exploring a full landscape is impossible [W3].

**WATCH OUT**

Do not assume "higher number = better" for every dataset. Read each dataset's README before using the `target` column. For Amylase, IRED and Hydro the descriptions say activity or stability, but the direction must be confirmed when we load the data.

## 2.3 Why a prediction model is needed: the surrogate

We cannot afford to measure every variant, so we train a model on the few we have measured and use it to guess the rest. This model goes by several names that all mean the same thing in our project: **surrogate**, **proxy**, **oracle** or **learned reward model**. The number the search tries to maximise is the **reward**.

**VERIFIED FACT** [9] [W12]

Offline model-based optimization papers describe the same danger. A simple gradient-ascent optimizer gets "fooled" by erroneously high predictions on inputs far from the training data [W12]; this is why Conservative Objective Models were proposed [9].

## 2.4 Machine-learning basics you need

### Regression, features and honest splitting

**Regression** means predicting a number from inputs (features). You fit the model on **training** data, tune it on **validation** data and report results on **test** data it has never seen. A model that does well on training data but badly on new data is **overfitting**.

### Embeddings from a protein language model (ESM-2)

A regression model needs numbers, not letters. **ESM-2** is a family of protein language models; its six sizes range from 8 million to 15 billion parameters [4][W10]. You pass a sequence through it and get a vector (an **embedding**) that summarises the sequence. The vector length depends on the model: 320 for the 8M model, 480 for 35M and 640 for 150M [W10b].

"**Frozen**" means we only run ESM-2 to read out vectors. We never update its weights. Each sequence gives one vector per residue; we average them into one vector per sequence (mean pooling, our choice). We compute the vectors once and save them.

**VERIFIED FACT** [3][W1]

FLIP2 compared models on its splits and reports that simple ridge regression on one-hot sequences often matched or outperformed fine-tuned protein language models. This is why we do not fine-tune anything.

### Ridge regression, small neural networks and ensembles

**Ridge regression** is linear regression with a penalty on large weights. A small **multilayer perceptron (MLP)** is a neural network with a couple of hidden layers. An **ensemble** is a group of models trained on slightly different data (for example bootstrap resamples). The average of their predictions is the predicted fitness

μ(x). The **spread** (standard deviation) of their predictions is a measure of uncertainty σ(x): where the members disagree, the model is unsure [16].

### Spearman rank correlation

Spearman correlation measures how well the **ranking** of predictions matches the ranking of true values. It is the Pearson correlation of the ranks, runs from −1 to +1, and does not need a straight-line relationship [W5]. FLIP2 reports Spearman and NDCG on held-out test sets [W1]. For us, ranking matters more than exact values because we want to pick the best variants.

## 2.5 Uncertainty, calibration and conformal prediction

A prediction with no error bar is hard to trust. A **prediction interval** gives a range that should contain the true value. **Calibration** asks whether the intervals are honest: a 90% interval should contain the true value about 90% of the time. The fraction of cases it actually contains is called **coverage**.

**Conformal prediction** is a wrapper that turns any point predictor into intervals with a chosen coverage level, without assuming a specific distribution [15][W6]. The simple "split" recipe is:

1. Split the labelled data into a training part and a calibration part.
2. Fit the model on the training part.
3. Compute the absolute errors on the calibration part.
4. Take a high quantile of those errors (about the 90th percentile for a 90% target; the exact rule adds a small finite-sample correction [15]).
5. The interval for a new point is prediction ± that error.

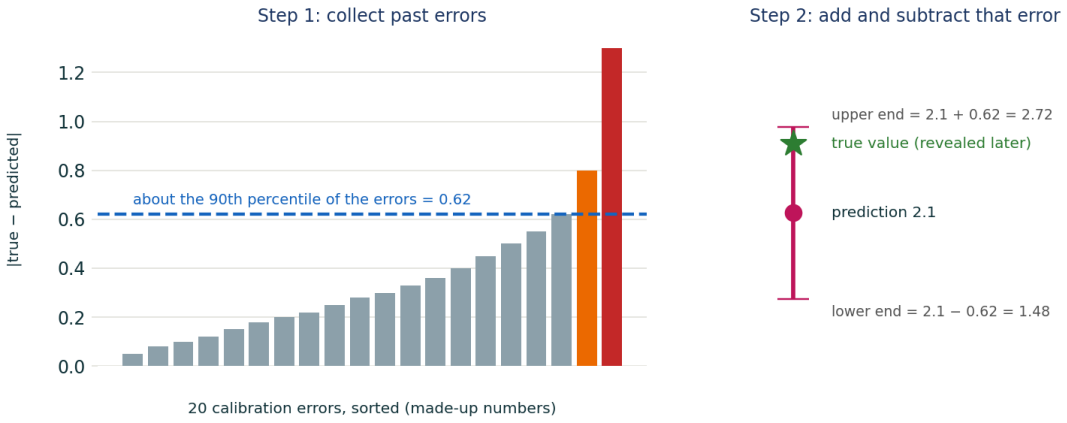



*Figure 2.1 Conformal intervals in two steps. All numbers are made up to illustrate the recipe.*

**WATCH OUT**

The guarantee holds only if calibration data and new points are **exchangeable** (roughly: drawn from the same distribution) [15]. A search that climbs toward high predicted reward breaks this on purpose. So we never assume 90% coverage holds during the search. We **measure** real coverage on each revealed batch, and a drop is one of our failure signals.

## 2.6 Searching with a limited label budget

The setting is called **sequential optimization** (or active learning). The vocabulary:

| Term | Meaning here |
|---|---|

| | |
|---|---|
| **Pool** | All variants that exist in the dataset. They are all "measured" already, but we pretend we do not know the labels. |
| **Seed set** | The small set of labels we are allowed to see at the start. |
| **Query** | Asking the pretend-lab for the true label of a chosen variant. |
| **Batch / round** | We query several variants at once (a batch). One cycle of choose, reveal, retrain is a round. |
| **Label budget** | The total number of labels we may reveal. Every method gets the same budget. |

**Exploration vs exploitation.** Exploiting means picking what the model currently thinks is best. Exploring means picking uncertain variants to learn more. Good search balances both.

## The search methods we compare

| Method | One-line idea | Source |
|---|---|---|
| **Random search** | Pick variants at random. The sanity-check baseline. | - |
| **Greedy** | Pick the variants with the highest predicted reward. | - |
| **Genetic algorithm** | Keep a population, select fitter parents, combine them (crossover), randomly change them (mutation) and repeat [W7]. | [W7] |
| **AdaLead-style** | A simple adaptive greedy search that beat more complex methods on several sequence design tasks. | [12] |
| **Bayesian optimization (BO)** | Fit a surrogate, score points with an acquisition function (for example expected improvement), sample where that score is best [14]. | [14] |
| **Reinforcement learning (optional)** | An agent builds a variant through mutation steps and receives the reward. DyNA-PPO is a published example [11]. | [11] |

> **VERIFIED FACT** [14]
> Bayesian optimization is, in its classic form, best suited to continuous domains of fewer than 20 dimensions [14]. Our inputs are discrete variants with embeddings of hundreds of dimensions, so our BO baseline will use ensemble mean and spread for its acquisition scores on the discrete pool and will not use a Gaussian process. That choice is ours.

## Regret: how we score a search

**Regret** measures how far a search is from the best possible outcome [W11]. Our definitions, which are design choices for this project:

- **Simple regret** = (best fitness in the landscape) − (best fitness found so far). Lower is better.

- **Cumulative regret** = the sum, over all queried variants, of (best fitness in the landscape) − (that variant's fitness).
- **Top-1% recall** = the fraction of the landscape's top 1% of variants that the search found.

## 2.7 Reward hacking: why this project exists

**Goodhart's law** says that when a measure becomes a target it stops being a good measure [W8]. In machine learning this appears as **reward overoptimization**: pushing hard on a learned proxy can raise the proxy score while true performance gets worse [5]. In sequence design, oracle models that differ only in architecture or training seed produced conflicting rankings of the same 12 design methods, with poor generalization outside the training data named as a key issue [6].

**KEY IDEA**

All the existing remedies we reviewed use a fixed recipe: a penalty, a prior, a conservative training rule or periodic retraining. They do not ask *why* the reward failed this time. Our bet is that a diagnosis-dependent, verified repair may recover better. It is a bet to be tested, not a result.

## 2.8 LLMs, JSON and validation

A large language model (LLM) can turn a sentence such as "maximise fitness but avoid risky, far-away mutations" into a structured **JSON** form. LLMs sometimes return invalid JSON, wrong types or invented keys, so output must be validated before use [W9]. **Pydantic** is a Python library that defines a schema, generates its JSON schema and validates data against it; failed validation can be fed back to the model for a retry [W9].

**OUR DESIGN CHOICE**

Our LLM has exactly two jobs: fill the reward form, and choose one repair from the menu with a parameter inside its allowed range. It never scores variants, never writes code and never sees unrevealed labels. Output is validated, retried once, then replaced by a rule-based default. An ablation (a run with the LLM removed) tests whether it adds anything.

**VERIFIED FACT** [7][8]

LLM-written rewards already exist: Eureka writes reward code and reported beating human experts on 83% of 29 tasks [7]; Text2Reward generates dense reward code from a goal and supports human-feedback refinement [8]. Both target robotics, which is why our pitch is the **checked** LLM role and not "LLMs can write rewards".

## 2.9 Distribution shift and the FLIP2 splits

**Distribution shift** means the test data differ from the training data. Protein engineering is full of it: later rounds explore more mutations, new positions, higher fitness. FLIP2 builds 16 train-test splits in five categories to mimic this [W1].

| Split type | Train on… | Test on… | Difficulty (per FLIP2) |
|---|---|---|---|
| **Number** | fewer mutations | more mutations | easier |
| **Mutation** | some mutations | different mutations | easier |
| **Position** | some positions | other positions | among the hardest |
| **Fitness** | lower fitness | higher fitness | among the hardest |
| **Wild type** | some wild-type proteins | another wild-type protein | among the hardest |

> **VERIFIED FACT** [W1]
> FLIP2 reports that wild-type, position and fitness splits are much more challenging than number and mutation splits, and that these splits are more challenging than random splits with equal training size.

**Why this matters to us:** a search starting from a shifted seed set must travel into the region where the model has no data. That is exactly where we expect proxy failures, so these splits give us natural failure scenarios on real measurements.

PART 3

# The method in detail

Exactly what our system does, one piece at a time

## 3.1 The reward, term by term

In round t the reward for a candidate variant x is built from approved terms. This formula is **not** our contribution; uncertainty penalties, conservative objectives and diversity terms already exist [9][11].

$$R_t(x) = \mu_t(x) - \lambda_t \cdot \sigma_t(x) - \rho_t \cdot d_t(x) - \eta \cdot \Sigma_j \max(0, g_j(x)) + \beta_t \cdot D(x, B_t)$$

| Symbol | Name | Plain meaning | Protects against |
|---|---|---|---|
| **μ(x)** | Predicted fitness | Ensemble average prediction | Nothing; it is the thing we want high |
| **σ(x)** | Uncertainty | Ensemble spread; high means "models disagree" | Trusting a guess nobody agrees on |
| **d(x)** | Distance from known data | Distance from x to the nearest labelled variant in embedding space, scaled to 0–1 with the labelled set | Wandering into regions with no data |
| **g_j(x)** | Hard constraints | For example "at most k mutations". Positive value means violated | Candidates that break the rules |
| **D(x, B_t)** | Diversity | How far x is from variants already picked in this batch | Ten near-identical picks |
| **λ, ρ, η, β** | Weights | Set in the specification, each with an allowed range | They are what repairs adjust |

A batch is built greedily: each next variant is the one with the highest reward given the variants already picked for the batch.

## 3.2 The reward specification and the validator

The reward lives in a strict JSON form. The LLM fills it in from a plain-language goal; the validator rejects anything with unknown terms or weights outside their ranges. The weights below are placeholders to show the format.

```
{
  "goal": "maximize_fitness",
  "terms": {
    "uncertainty_penalty": {"weight": 0.5, "allowed_range": [0.0, 2.0]},
    "distance_penalty":    {"weight": 0.3, "allowed_range": [0.0, 2.0]},
    "diversity_bonus":     {"weight": 0.1, "allowed_range": [0.0, 1.0]}
  },
  "hard_constraints": {"max_mutations": 4},
  "budget": {"rounds": 10, "batch_size": 10},
  "abstain_after_failed_repairs": 2
}
```

**OUR DESIGN CHOICE**

Planned LLM rules: temperature 0, JSON output, one retry on invalid output, then a rule-based default. Every prompt, response and random seed is logged. The LLM sees only the goal text and summary statistics of batches that were already revealed.

## 3.3 The five failure detectors

After each batch is revealed, five checks run. A check that fails creates a **counterexample record**: its type, the evidence and the round. The numbers below are **starting values** that we tune only on the development landscape.

| Type | Plain idea | Planned trigger | Allowed repairs |
|---|---|---|---|
| **Constraint violation** | A rule was broken | A selected variant has $g_j(x) > 0$. Because constraints are applied as a filter first, this mainly tests the filter and any edited specification. | Reject variant, fix the filter, keep the old reward |
| **Support shift** | The search left the region the model knows | At least 3 variants in the batch fall outside their conformal interval AND have $d(x)$ above the 90th percentile of the labelled set | Shrink the trust radius (max distance from labelled variants); raise $\rho$ |
| **Miscalibration** | Error bars are too optimistic | Coverage over the last 30 revealed labels (3 batches) is below 0.75, against a 0.90 target | Refit the conformal quantile and retrain; raise $\lambda$ |
| **Model disagreement** | Ensemble members rank candidates differently | Mean pairwise Spearman among members on the top 100 candidates is below 0.5 | Use the median or a lower quantile of members; raise $\lambda$ |
| **Diversity collapse** | The batch is near-duplicates | Mean pairwise Hamming distance in the batch is below 2 | Raise $\beta$ or use farthest-point selection |

**VERIFIED FACT** our calculation

Why 30 labels and a 0.75 trigger? If true coverage is 0.90, coverage measured on 30 labels has a standard deviation of $\sqrt{(0.9 \times 0.1 / 30)} \approx 0.055$. A trigger at 0.75 is about $(0.90 - 0.75) / 0.055 \approx 2.7$ standard deviations below, so false alarms should be rare. A single batch of 10 would be too noisy.

If several detectors fire in one round they are handled one at a time in this order: constraint violation, support shift, miscalibration, model disagreement, diversity collapse. The system re-checks after each repair.

**WATCH OUT**

Conflict between several measured properties is out of scope, because each chosen dataset reports a single property. If the professor asks about "objective conflict", say that clearly.

## 3.4 Minimal repair, verification, rollback and abstention

**Repair cost.** Let $\theta$ be the vector of reward parameters. Changing reward $R$ into $R'$ costs $C(R', R) = \Sigma_j \, |\theta'_j - \theta_j| \, / \, \text{range}_j + c \times$ (number of terms changed). This makes "smallest repair" a number we can measure. The constant $c$ is still to be chosen.

The system tries the repairs allowed for the diagnosed type from cheapest to most expensive and keeps the first that passes **four checks**, all computed from labels that are already revealed:

1. If the round were replayed under the new reward R′, the counterexample variants would not be picked again.
2. All hard constraints still hold.
3. Coverage on the calibration split is within a set tolerance of the target.
4. Rank correlation on earlier labelled variants does not drop by more than a set margin.

If no repair passes, the old reward is kept (**rollback**). After two failed repairs in a row the system **abstains**: it stops and asks for more labels. Every step goes into the **audit log**.

## 3.5 A worked example

> **TRY IT**
>
> **The numbers here are made up to explain the steps. They are not results.**
>
> After round 5, 21 of the last 30 revealed labels fall inside the 90% interval, so coverage is 0.70, below the 0.75 trigger. In the newest batch, 3 variants are outside their interval and all three have d(x) above the 90th percentile. Support shift and miscalibration both fire.
>
> Support shift is handled first. Shrinking the trust radius from 4 to 3 mutations is the cheapest allowed repair; it passes all four checks, so it becomes the new reward. After the next batch, the miscalibration detector is checked again. If coverage is still low, the conformal quantile is refit. Each step is written to the audit log.

## 3.6 The algorithm in ten steps

1. Compile the user's goal into a reward specification with the LLM and validate it. Stop if it is rejected.
2. Embed all variants once, draw the seed set and reveal its labels.
3. Train the ensemble and calibrate the conformal intervals.
4. Score the pool with R_t and pick a batch of B variants greedily with the diversity term.
5. Reveal the labels of the batch and add them to the labelled set.
6. Run the five detectors. If none fires, retrain and continue with R_t.
7. For each fired detector, store a counterexample record with its type and evidence.
8. Take the allowed repairs for that type and check them in order of cost.
9. Accept the first repair that passes verification as the new reward. Otherwise keep the old one. Abstain after two failed repairs in a row.
10. Append version, trigger, repair, cost and outcome to the audit log, and repeat until the label budget is used up.

## 3.7 Suggested code layout

This is one possible way to split the work between us. It is a design suggestion, not a requirement.

| Module | Responsibility |
|---|---|
| `embed.py` | Run ESM-2 once over all sequences and cache the vectors |
| `surrogate.py` | Ensemble of ridge and MLP members; bootstrap; μ(x) and σ(x) |
| `conformal.py` | Split-conformal quantile; coverage on a batch |

| | |
|---|---|
| `reward.py` and `validator.py` | Reward formula; JSON schema (Pydantic) and range checks |
| `llm_interface.py` | Compile the goal; propose a repair; retry; fallback |
| `detectors.py` | The five checks and the counterexample record |
| `repairs.py` and `verifier.py` | Repair library, cost, four verification checks, rollback, abstain |
| `search.py` | Greedy and baseline searches (random, genetic, AdaLead-style, BO) |
| `audit.py` | Versioned log in JSON Lines or SQLite |
| `evaluate.py` | Hidden-label oracle, metrics, seeds, statistics, plots |

PART 4

# Data and experiments

How we test the idea fairly, using public data where the truth already exists

## 4.1 The datasets

We use public experimental fitness data from the FLIP2 benchmark, plus optional extras. FLIP2 CSV files have four columns: `sequence`, `target` (the measured fitness), `set` (train, test or validation) and `validation` (True or False) [W1].

| Landscape | Facts from the FLIP2 page [W1] | Split we plan to use | Role |
|---|---|---|---|
| **Alpha amylase** | 3,706 variants, 488 single mutants, up to 8 mutations; stain-removal activity of Bacillus subtilis alpha amylase; MIT license | close-to-far (position): train on mutations within 7.3 Å of the active site | Development: tune thresholds here only |
| **Imine reductase (IRED)** | 17,143 variants, 1,207 single mutants, up to 15 mutations; activity from a microfluidic screen; CC-BY 4.0 | two-to-many (number): train on 0, 1 or 2 mutations | Test |
| **Hydrophobic core (Hydro)** | 24,935 variants of three proteins (seven core residues randomised to hydrophobic amino acids); stability; CC-BY 4.0 | low-to-high (fitness): train below median fitness, test above | Test |
| **GB1 (original FLIP)** | Domain stability and binding [W1][2] | low_vs_high | Optional fourth landscape |
| **A ProteinGym assay** | ProteinGym has more than 250 standardized assays [1] | To be chosen | Optional extra check |

**VERIFIED FACT** [W1], our sum

The three main landscapes together contain 3,706 + 17,143 + 24,935 = 45,784 sequences to embed.

**OUR DESIGN CHOICE** [W1]

We deliberately skip two FLIP2 datasets. **NucB** stores its target as four activity bins and not a continuous value. **TrpB** has 228,298 variants, which is too large for our compute. The Rhodopsin and PDZ3 sets are small or built from variant pairs.

**OPEN DECISION**

Hydro contains three different wild-type proteins. Do we run each one as a separate pool, or all together? Also decide whether GB1 or a ProteinGym assay is worth adding in Phase II.

## 4.2 The hidden-label simulation

We have no wet lab, but the datasets already contain a measured label for every variant. So we **pretend**. The optimizer is only allowed to see a label after it has queried that variant. This is a **retrospective** experiment.

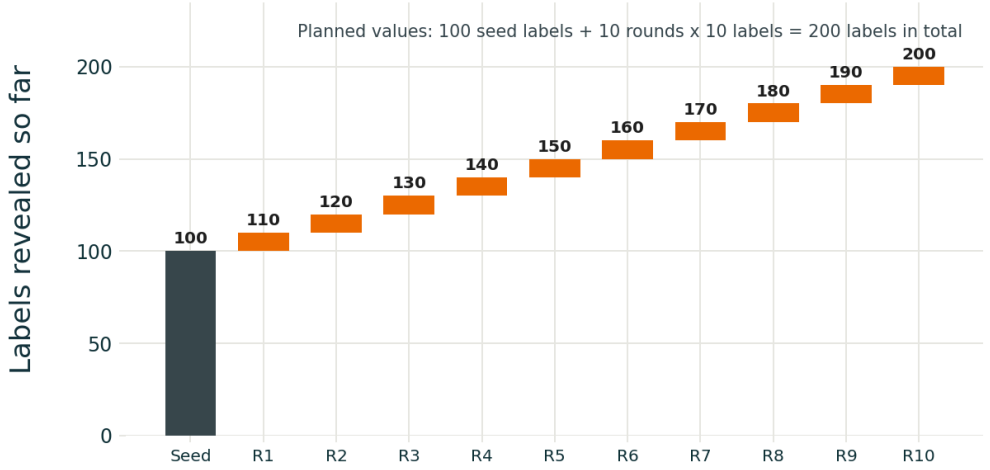



*Figure 4.1 Labels revealed during one planned run (100 seed + 10 rounds of 10).*

1. Load a landscape. The pool is every measured variant. Hide all labels.
2. Build the seed set from the training side of a shift split and reveal its labels (planned: 100).
3. Train the surrogate. Pick a batch (planned: 10) using the reward.
4. Reveal the true labels of that batch only.
5. Run the detectors, repair if needed, retrain.
6. Repeat for the planned 10 rounds. Every method gets the same seed set and the same budget.

| Landscape | Pool size | Labels used (planned 200) | Share of the pool |
|---|---|---|---|
| **Alpha amylase** | 3,706 | 200 | 5.4% |
| **IRED** | 17,143 | 200 | 1.17% |
| **Hydro** | 24,935 | 200 | 0.80% |

*Planned values (100 seed labels, 10 rounds, batch of 10, 10 random seeds per setting) are starting points that we can change.*

> **KEY IDEA**
> Seeding from the training side of a shift split is the trick. For IRED we start with variants that have 0 to 2 mutations. To find the best variants the search must move to many more mutations, which is where the model has no data and failures are expected.

## 4.3 Injected failures: testing the detectors

Natural shifts are not enough to measure detector precision and recall, because we do not know the "true cause" there. So we also create failures on purpose with a known cause.

| We break… | By… | The detector that should fire |
|---|---|---|
| **Support** | Using a seed set limited to variants with at most 2 mutations | Support shift |
| **Calibration** | Using a very small calibration set | Miscalibration |

| | | |
|---|---|---|
| **Constraints** | Disabling the mutation-count filter | Constraint violation |
| **Diversity** | Removing the diversity term | Diversity collapse |

*Model disagreement has no injection listed in the plan yet.*

**OPEN DECISION**

Decide how to inject **model disagreement** (for example, by making ensemble members much more different from each other) so that all five detectors can be evaluated.

## 4.4 Baselines and ablations

| Baselines (other methods) | Ablations (our method with something removed) |
|---|---|
| Random search | No LLM: a rule-based repair selector instead |
| Greedy on predicted fitness | No verifier |
| Genetic algorithm | No rollback |
| AdaLead-style search [12] | No abstention |
| Bayesian optimization (ensemble-based) | One universal penalty instead of type-specific repairs |
| Fixed uncertainty penalty | Each repair removed in turn |
| Periodic retraining with no diagnosis | Optional: PPO as an extra baseline |

## 4.5 Metrics in plain words

All metrics are computed against the **hidden true labels**, never against the reward the optimizer used. Judging the optimizer by its own reward would be circular.

| Metric | Plain meaning | Why we use it |
|---|---|---|
| **Best-so-far true fitness** | The best real fitness the search has found, rescaled with the landscape's min and max (for reporting only) | The headline outcome |
| **Simple / cumulative regret** | Gap to the best possible; the summed gap over all queries | Standard for sequential search [W11] |
| **Top-1% recall** | Share of the true top 1% variants found | How many good variants we actually found |
| **Interval coverage and width** | How often the 90% interval contains the truth, and how wide it is | Is the uncertainty honest? |
| **Batch Spearman** | Rank agreement between predicted and true fitness inside a batch [W5] | Is the model still ranking well? |
| **Detector precision / recall / F1** | Of the alarms raised, how many were right; of the real failures, how many were caught | Do the detectors work? |
| **False-alarm rate** | Alarms on runs with no injected failure | Avoid needless repairs |
| **Rounds to recovery** | Rounds after the first repair until coverage is within 0.05 of target and the batch's mean true fitness is no lower than before the failure | Our signature metric |

| | | |
|---|---|---|
| **Repairs, repair cost, rollbacks, abstentions** | How much the system had to change and how often it gave up | Is the repair really "minimal"? |
| **Constraint violations, diversity** | Rule-breaking rate; mean pairwise Hamming distance of batches | Quality of the batches |
| **LLM calls and run time** | Cost of the approach | Practicality |

## 4.6 Statistics

- **10 random seeds** per setting, reported as the mean with a **95% bootstrap confidence interval**.
- **Paired Wilcoxon signed-rank tests** across seeds, since every method shares the same seed sets.
- Results reported **per landscape and per split**, not only as one average.
- Tune thresholds on Amylase only. Test landscapes are never used for tuning.

## 4.7 Rules of honest science for this project

WATCH OUT

**Label leakage.** The optimizer must never see an unqueried label. Not for training, not for threshold tuning, not for the LLM.

**Circular evaluation.** Score on the hidden true fitness, not on the reward.

**Tuning on the test set.** Thresholds are chosen on Amylase and then frozen.

**Cherry-picking.** Report every landscape and split we run, including bad ones.

**Overclaiming.** We find variants in a pool of already-measured data. We do not design proteins.

**A negative result is fine.** If plain Bayesian optimization matches our method, the failure analysis and the protocol are still the contribution.

PART 5

# Plan and first steps

An 18-week schedule, a learning order, and what to do this week

## 5.1 The 18-week schedule

The template's deliverable table ends with the final report in Week 18, so we planned backwards from that. Diamonds mark deliverables.

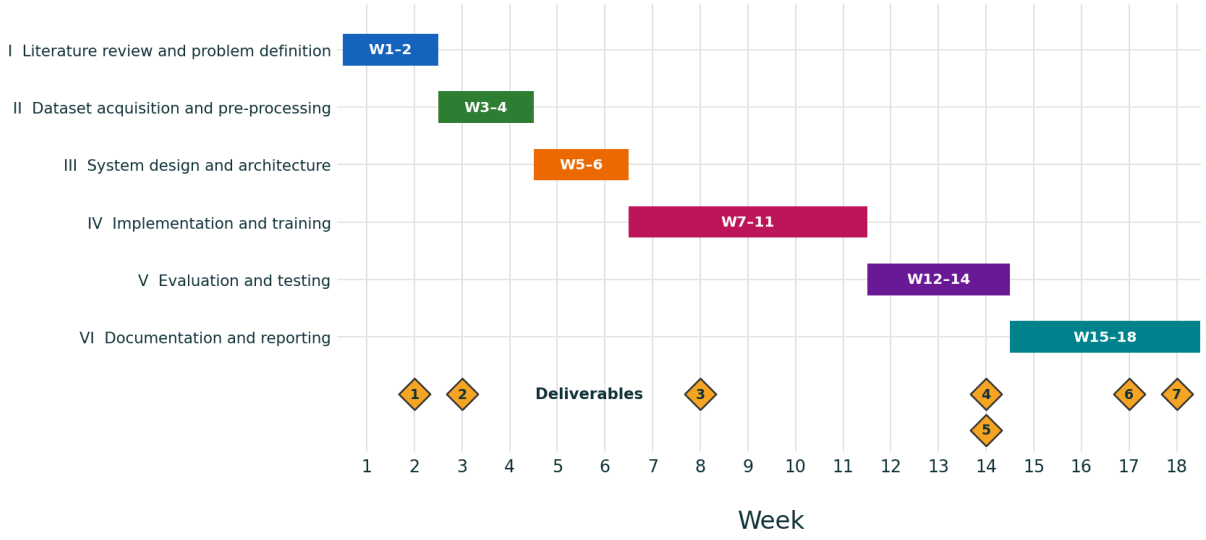



*Figure 5.1 Phase plan (Weeks 1 to 18). Deliverables: 1 Synopsis (W2), 2 Literature review report (W3), 3 Working prototype / baseline (W8), 4 Final model and code (W14), 5 Evaluation report (W14), 6 Paper draft (W17), 7 Final report and presentation (W18).*

| Phase | Weeks | You are done when… |
|---|---|---|
| **I Literature and problem definition** | 1–2 | You can explain the problem, the five failure types and the scope without notes |
| **II Data and pre-processing** | 3–4 | Amylase, IRED and Hydro are loaded, cleaned, explored, and ESM-2 vectors are cached |
| **III System design** | 5–6 | Reward schema, validator, detector rules, repair library and audit-log format are written down and agreed |
| **IV Implementation** | 7–11 | Ensemble, conformal calibration, baselines, detectors, repairs, verifier and LLM interface run end to end; thresholds tuned on Amylase |
| **V Evaluation** | 12–14 | All methods run for 10 seeds on the shift splits and injected failures; ablations and statistics done |

| | | |
|---|---|---|
| **VI Reporting** | 15–18 | Report, paper draft and presentation delivered |

## 5.2 Learning order

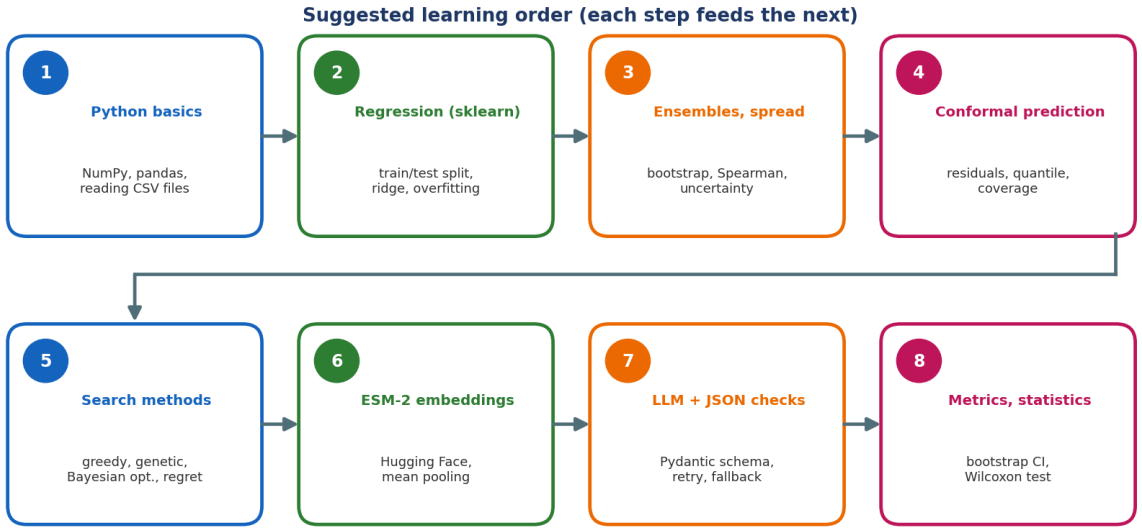



*Figure 5.2 Suggested learning order. Learn each step by using it on a small dataset, not by only reading.*

## 5.3 This week: reproduce the failure on a tiny scale

The best way to understand the project is to see the problem with your own eyes before building anything. The goal is to show, on real data, that greedily picking the highest predictions gives worse prediction errors than picking at random.

1. Download one FLIP2 split (for example Amylase) and load it with pandas.
2. Embed the sequences once with the 8M ESM-2 model and save the vectors.
3. Train a 5-member ridge ensemble on 100 random labelled variants.
4. Check whether the 90% conformal intervals really cover about 90% of held-out variants.
5. Take the 10 variants with the highest predicted fitness. Reveal their labels. Compare their prediction errors with 10 randomly chosen variants.
6. Write down what you see. Do not worry if the effect is small; note it either way.

**Loading a split (the path pattern follows the example on the FLIP2 page; check the Download table for exact file names):**

```
import pandas as pd
df = pd.read_csv('assets/splits/amylase/one_to_many.csv.gz')   # pandas reads .gz directly
train_df = df[df['set'] == 'train']
test_df  = df[df['set'] == 'test']
```

**Embedding sketch with ESM-2 8M (untested here; checkpoint name and sizes are from the Hugging Face model card [W10]):**

```
import torch
from transformers import AutoTokenizer, AutoModel
tok   = AutoTokenizer.from_pretrained("facebook/esm2_t6_8M_UR50D")
model = AutoModel.from_pretrained("facebook/esm2_t6_8M_UR50D").eval()
```

```
seqs = train_df["sequence"].tolist()[:32]          # do this in batches
with torch.no_grad():
    b   = tok(seqs, return_tensors="pt", padding=True)
    out = model(**b).last_hidden_state                # (n, tokens, 320)
    m   = b["attention_mask"].unsqueeze(-1)
    emb = (out * m).sum(1) / m.sum(1)                 # mean over tokens
```

**OPEN DECISION**

The mean above also includes the special start and end tokens. Decide together whether to exclude them and keep the choice the same for every sequence.

**Ensemble, uncertainty and conformal interval (this sketch ran correctly on random test data; it has not been run on real protein data yet):**

```
import numpy as np
from sklearn.linear_model import Ridge
rng = np.random.default_rng(0)

def fit_ensemble(X, y, M=5, alpha=1.0):
    models = []
    for _ in range(M):
        idx = rng.integers(0, len(X), len(X))        # bootstrap sample
        models.append(Ridge(alpha=alpha).fit(X[idx], y[idx]))
    return models

def predict(models, X):
    preds = np.stack([m.predict(X) for m in models])
    return preds.mean(axis=0), preds.std(axis=0)     # mu(x), sigma(x)

def conformal_q(y_cal, mu_cal, level=0.9):
    err = np.sort(np.abs(y_cal - mu_cal)); n = len(err)
    k = int(np.ceil((n + 1) * level))                # finite-sample rule
    return err[min(k, n) - 1]

def coverage(y, mu, q):
    return float(np.mean(np.abs(y - mu) <= q))
```

## 5.4 Risks and how we handle them

| Risk | Why it matters | Our answer |
|---|---|---|
| **Detectors are noisy with batches of 10** | A false alarm triggers an unnecessary repair | Use the last 30 labels for coverage; tune on Amylase; report the false-alarm rate |
| **Conformal coverage is not guaranteed during the search** | The search itself breaks exchangeability [15] | Measure real coverage per batch and treat a drop as a signal |
| **The LLM adds nothing, or returns invalid JSON** | The LLM would be decoration | Validator, one retry, rule-based fallback, and the no-LLM ablation |
| **Large landscapes are too slow** | Embedding and repeated retraining cost compute | Use only Amylase, IRED and Hydro; cache embeddings; start with the 8M model |
| **Plain Bayesian optimization matches our method** | No measurable gain in fitness | Report it honestly; the contribution is then the failure analysis and protocol |
| **Assay noise looks like a failure** | One noisy label may be mistaken for a counterexample | Use interval-based rules over several labels |

| | | |
|---|---|---|
| **Thresholds overfit one dataset** | Results would not generalize | Tune on Amylase only and report on the others |

## 5.5 Tools

| Category | Planned |
|---|---|
| **Language** | Python 3.10 or later |
| **Libraries** | PyTorch; Hugging Face Transformers (ESM-2); scikit-learn; NumPy; pandas; SciPy; Pydantic; matplotlib; optional MAPIE and Stable-Baselines3 |
| **Environment** | Google Colab (T4 GPU for the one-time embedding step) and VS Code with Jupyter |
| **Storage** | Parquet or HDF5 for cached embeddings; JSON Lines or SQLite for the audit log |
| **LLM** | An instruction-following model through an API with JSON output at temperature 0 |
| **Tracking** | Git / GitHub; Weights & Biases (optional) |

PART 6

# Honesty check

What is sourced, what is our choice, what is still open

## 6.1 What we verified

| Claim | Where it comes from |
|---|---|
| ProteinGym has more than 250 standardized DMS assays and supports fitness prediction and design | [1] |
| FLIP2: 7 new datasets, 16 splits in 5 categories, ICML 2026, data under CC-BY 4.0 or MIT; dataset sizes and split definitions in Part 4 | [W1] |
| Wild-type, position and fitness splits are much harder than number and mutation splits; simple ridge regression often matches or beats fine-tuned protein language models | [W1] |
| ESM-2 comes in six sizes from 8M to 15B parameters; embedding length 320, 480 and 640 for the 8M, 35M and 150M models | [4] [W10] |
| Optimizing a learned reward model too far can hurt the true objective (Goodhart's law) | [5] [W8] |
| 12 sequence design methods gave conflicting rankings under different oracles | [6] |
| Conservative objective models, DyNA-PPO, AdaLead and Proximal Exploration exist and use fixed remedies | [9] [11] [12] [13] |
| Classic Bayesian optimization is best suited to continuous domains below 20 dimensions | [14] |
| Conformal guarantees assume exchangeability; ensemble spread can serve as uncertainty | [15] [16] |
| LLM reward generation exists for robotics (Eureka: 83% of 29 tasks; Text2Reward) | [7] [8] |

## 6.2 What is our design choice

| Item | Planned value or rule |
|---|---|
| **Reward form** | $\mu - \lambda\sigma - \rho d - \eta\cdot$(constraint violations) $+ \beta\cdot$diversity |
| **Surrogate** | 5 to 10 members (ridge and a small MLP, 256 and 128 units, ReLU, Adam, lr 1e-3, weight decay 1e-4, early stopping) on bootstrap samples |
| **Embeddings** | ESM-2 8M first (320-d), larger only if time allows; mean pooling; frozen |
| **Conformal** | 90% target; 20% of revealed labels held out for calibration |

| | |
|---|---|
| **Budget** | 100 seed labels, 10 rounds, batch of 10, 10 random seeds |
| **Detector thresholds** | 0.75 coverage on last 30 labels; 3 variants outside interval with d(x) above the 90th percentile; Spearman below 0.5 on top 100; Hamming below 2 |
| **Handling order** | constraint, support shift, miscalibration, disagreement, diversity |
| **Abstain rule** | after two failed repairs in a row |
| **LLM rule** | temperature 0, one retry, rule-based fallback, full logging |
| **Search restriction** | Genetic and AdaLead-style baselines only propose pool variants within Hamming distance 2 of the current best |

## 6.3 What is still open

- Hydro: separate pools per wild type, or one pool?
- Whether to add GB1 or a ProteinGym assay as a fourth landscape.
- How to inject **model disagreement** for detector testing.
- The value of the repair-cost constant **c** and the verification tolerances.
- Which LLM and API to use.
- Whether mean pooling should exclude the special start and end tokens.
- The direction of the `target` column for each dataset (confirm from the README).
- Whether our professor requires RL as the main method.
- The guide's name and signature for the declaration page.

## 6.4 Updates to consider in the submitted synopsis

These come from comparing the submitted document with the sources checked for this guide.

- **Datasets.** Table 5.1 lists ProteinGym (one development and one test landscape), FLIP GB1 and "one or two" FLIP2 landscapes. Part 4 here proposes Amylase for development and IRED plus Hydro for testing. If you agree, make the two documents match.
- **FLIP2 wording.** Section 3.1 calls FLIP2 a preprint. The FLIP2 site states it appeared at ICML 2026 as a spotlight, so reference [3] can mention that as well as bioRxiv.
- **Conformal caveat and numbers.** Section 5.3 does not mention that conformal coverage can break during search, and Table 5.2 says "a set margin" where this guide has planned starting values (Part 3.3).
- **AI disclosure.** The disclosure line says all cited references were checked against their pages. Keep a statement like that only if it is true for what you actually did.

PART 7

# Viva prep, glossary and sources

Questions to rehearse, words to know, and what to read

## 7.1 Likely questions and grounded answers

**Q. What problem does your project solve?**

A search that maximises a learned fitness predictor can exploit its errors, so predicted reward rises while measured fitness does not [5][6]. We detect when that happens, work out the type of failure, repair the reward minimally and verify the repair.

**Q. Why not just add an uncertainty penalty?**

That is one of our baselines. It uses a fixed recipe whatever the cause of failure. We test whether diagnosis-dependent repair recovers better. It might not, and we would report that.

**Q. Is it novel?**

Each ingredient exists. In the papers we reviewed we did not find a protocol that records contradictions as typed counterexamples, picks the smallest verified repair and measures recovery. That is a statement about our review, not a proof of absence.

**Q. What do you actually train?**

The surrogate ensemble on frozen ESM-2 embeddings, retrained after repairs. The LLM is not trained. Detectors and repairs are rule-based first.

**Q. Where is reinforcement learning?**

Optional extra baseline (a PPO policy in the style of DyNA-PPO [11]). It is not the core method. We report its result whether or not it wins.

**Q. What does the LLM do and why is it safe?**

It fills a strict JSON form and chooses a repair from a fixed menu. A validator checks everything, there is one retry and a rule-based fallback, and an ablation removes it to test its value.

**Q. How do you stop the optimizer from cheating with labels?**

Labels are hidden until queried. Thresholds are tuned on Amylase only. Scores use the hidden true fitness, not the reward.

**Q. Why FLIP2 and these three landscapes?**

FLIP2 provides position, number and fitness shift splits on real measurements [W1], which create the failures we want to study. Amylase, IRED and Hydro are small enough for our compute.

**Q. Why not fine-tune a protein language model?**

FLIP2 found simple ridge regression often matches or beats fine-tuned models [W1], and it would cost compute without helping the research question.

**Q. Conformal prediction promises 90%. Why do you expect it to fail?**

The guarantee needs exchangeability [15]. A search that climbs the reward breaks it, so we measure real coverage and treat a drop as a signal.

**Q. What does "minimal repair" mean?**

The repair with the lowest cost C, which sums the normalised parameter changes plus a charge for each term changed.

**Q. What if Bayesian optimization is just as good?**

Then that is our result. The failure analysis, detector evaluation and protocol remain valid findings.

**Q. Did you design a protein?**

No. We choose among variants that were already measured in public datasets. We make no claim about function or safety.

## 7.2 Glossary

| Term | Meaning |
|---|---|
| **Abstain** | Stop and ask for more labels when no repair can be verified. |
| **Acquisition function** | A score that decides which point to measure next in Bayesian optimization [14]. |
| **Amino acid** | A building block of proteins; 20 standard types, each with a one-letter code. |
| **Audit log** | A versioned record of every reward change, its trigger and outcome. |
| **Batch** | Several variants queried in one round. |
| **Calibration** | Whether stated confidence matches reality. |
| **Conformal prediction** | A wrapper that gives prediction intervals with a chosen coverage level [15]. |
| **Counterexample** | A queried variant whose true label contradicts what the reward expected. |
| **Coverage** | The fraction of true values inside the prediction intervals. |
| **Distribution shift** | Test data differ from training data. |
| **DMS** | Deep mutational scanning: measuring thousands of mutations in one experiment [W2]. |
| **Embedding** | A vector that summarises a sequence. |
| **Ensemble** | A group of models; their spread signals uncertainty [16]. |
| **Epistasis** | Mutation effects that do not simply add up [W3]. |
| **ESM-2** | A family of protein language models [4]. |
| **Exchangeability** | Roughly, data drawn from the same distribution in any order; the condition behind conformal guarantees. |
| **Fitness** | A measured number describing how well a variant performs a function. |
| **Fitness landscape** | The map from sequences to fitness [W2]. |

| | |
|---|---|
| **Frozen** | Model weights are not updated. |
| **Goodhart's law** | When a measure becomes a target, it stops being a good measure [W8]. |
| **Hamming distance** | Number of positions at which two equal-length strings differ [W4]. |
| **Label budget** | The total number of labels we may reveal. |
| **MLP** | Multilayer perceptron, a small neural network. |
| **Oracle / surrogate / proxy** | A learned model used in place of the real measurement. |
| **Pool** | All variants in the dataset that the search can choose from. |
| **Pydantic** | A Python library for defining and validating data schemas [W9]. |
| **Regret** | Gap between what a search found and the best possible [W11]. |
| **Retrospective** | Using data that already exists and hiding it, instead of running new experiments. |
| **Reward** | The score the optimizer tries to maximise. |
| **Reward overoptimization** | Raising the proxy score while true performance falls [5]. |
| **Seed set** | The labels revealed at the start. |
| **Spearman correlation** | Rank agreement between two lists [W5]. |
| **Variant** | A protein with one or more mutations. |
| **Wild type** | The natural, original sequence. |

## 7.3 Reading list in a good order

| Order | What | What to read in it |
|---|---|---|
| 1 | FLIP2 website [W1] and paper [3] | Overview, dataset cards, split types, key findings |
| 2 | Angelopoulos and Bates, conformal prediction [15] | The introduction and the split-conformal recipe |
| 3 | Frazier, Bayesian optimization tutorial [14] | The abstract and the first sections on surrogate and acquisition functions |
| 4 | Gao et al., reward overoptimization [5] | Abstract and the main figures |
| 5 | Surana et al., Overconfident Oracles [6] | Abstract and the discussion of oracle disagreement |
| 6 | BAIR blog on conservative objective models [W12] and [9] | Why optimizers get "fooled" off-distribution |
| 7 | DyNA-PPO [11] and AdaLead [12] | Method overview; these are the baselines we compare against |
| 8 | Eureka [7] and Text2Reward [8] | Abstracts: what LLM reward design already does |
| 9 | ESM-2 [4], ensembles [16], Proximal Exploration [13], Design-Bench [10] | Skim when needed |

## 7.4 Sources

### Papers (numbering matches the synopsis):

[1] P. Notin et al., "ProteinGym: Large-scale benchmarks for protein fitness prediction and design," NeurIPS Datasets and Benchmarks Track, vol. 36, 2023.

[2] C. Dallago et al., "FLIP: Benchmark tasks in fitness landscape inference for proteins," NeurIPS Datasets and Benchmarks Track, 2021.

[3] K. Didi et al., "FLIP2: Expanding protein fitness landscape benchmarks for real-world machine learning applications," ICML 2026; bioRxiv doi 10.64898/2026.02.23.707496.

[4] Z. Lin et al., "Evolutionary-scale prediction of atomic-level protein structure with a language model," Science, vol. 379, no. 6637, pp. 1123–1130, 2023.

[5] L. Gao, J. Schulman, J. Hilton, "Scaling laws for reward model overoptimization," ICML 2023, arXiv:2210.10760.

[6] S. Surana et al., "Overconfident oracles: Limitations of in silico sequence design benchmarking," arXiv:2502.17246, 2025.

[7] Y. J. Ma et al., "Eureka: Human-level reward design via coding large language models," ICLR 2024, arXiv:2310.12931.

[8] T. Xie et al., "Text2Reward: Reward shaping with language models for reinforcement learning," ICLR 2024, arXiv:2309.11489.

[9] B. Trabucco et al., "Conservative objective models for effective offline model-based optimization," ICML 2021, arXiv:2107.06882.

[10] B. Trabucco et al., "Design-Bench: Benchmarks for data-driven offline model-based optimization," ICML 2022, arXiv:2202.08450.

[11] C. Angermueller et al., "Model-based reinforcement learning for biological sequence design," ICLR 2020.

[12] S. Sinai et al., "AdaLead: A simple and robust adaptive greedy search algorithm for sequence design," arXiv:2010.02141, 2020.

[13] Z. Ren et al., "Proximal exploration for model-guided protein sequence design," ICML 2022.

[14] P. I. Frazier, "A tutorial on Bayesian optimization," arXiv:1807.02811, 2018.

[15] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," Found. Trends Mach. Learn., vol. 16, no. 4, 2023.

[16] B. Lakshminarayanan, A. Pritzel, C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," NeurIPS 2017, arXiv:1612.01474.

### Web pages used for background (accessed while preparing this guide):

[W1] FLIP2 benchmark website: https://flip.protein.properties/

[W2] Ranomics, deep mutational scanning overview: https://www.ranomics.com/deep-mutational-scanning-a-high-throughput-approach-to-mapping-protein-fitness-landscapes

[W3] Cell Systems 2023, learning protein fitness landscapes with DMS (epistasis, ruggedness): https://www.cell.com/cell-systems/fulltext/S2405-4712(23)00210-7

[W4] Hamming distance: https://www.ibm.com/think/topics/hamming-distance and https://www.datacamp.com/tutorial/hamming-distance

[W5] Spearman correlation: https://numiqo.com/tutorial/spearman-correlation

[W6] Conformal prediction: https://en.wikipedia.org/wiki/Conformal_prediction and https://www.stat.berkeley.edu/~ryantibs/statlearn-s23/lectures/conformal.pdf

[W7] Cornell Optimization Wiki, genetic algorithm: https://optimization.cbe.cornell.edu/index.php?title=Genetic_algorithm

[W8] Reward hacking and Goodhart's law guide: https://aisecurityandsafety.org/en/guides/reward-hacking/

[W9] Pydantic structured output: https://pydantic.dev/docs/ai/core-concepts/output/

[W10] ESM-2 sizes: https://www.nature.com/articles/s41598-025-05674-x ; checkpoints and embedding sizes: https://github.com/facebookresearch/ESM and https://huggingface.co/facebook/esm2_t6_8M_UR50D

[W11] Regret in Bayesian optimization (simple and cumulative): https://jmlr.org/papers/volume26/22-0523/22-0523.pdf

[W12] BAIR blog, conservative objective models: https://bair.berkeley.edu/blog/2021/10/25/coms_mbo/

*This guide was prepared with the help of an AI assistant (Perplexity) and checked against the sources above. Use it to learn, then confirm important facts in the original papers before quoting them.*